VOLUME 08 OF 10

Gaming Platforms and Hardware/Software Co-design

Upper undergraduate CE / capstone

Study boards, memory, GPU architecture, performance, input/display pipelines and integrated design.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 36 Embedded platforms, drivers and board design

36.1 Serial interfaces and peripheral contracts • 36.2 Firmware, RTOS scheduling and driver lifecycle • 36.3 Schematics, PCB layout, power integrity and signal integrity • 36.4 Bring-up, instrumentation and fault isolation

Module 37 GPU microarchitecture and memory systems

37.1 SIMT execution, divergence, occupancy and latency hiding • 37.2 Coalescing, local storage, caches and shared-memory banks • 37.3 Raster, texture, matrix and ray-tracing units • 37.4 CPU-GPU sharing, discrete memory and synchronization

Module 38 Quantitative performance engineering

38.1 Amdahl, roofline and bottleneck models • 38.2 Profilers, counters and trustworthy benchmark design • 38.3 SIMD, cache blocking, NUMA and job scheduling • 38.4 Energy, thermal behavior and sustained performance

Module 39 Gaming-platform constraints and interactive latency

39.1 Input-to-display latency and frame pacing • 39.2 Mobile and handheld systems: memory, power and lifecycle • 39.3 XR spaces, stereo views, pose timing and comfort constraints • 39.4 Portability, platform services and console-development boundaries

Module 40 Hardware/software co-design capstone

40.1 Workload characterization and measurable requirements • 40.2 Architecture alternatives, arithmetic precision and dataflow • 40.3 Interface design, virtualization and end-to-end evaluation • 40.4 Design review, evidence tiers and undergraduate CE completion

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 36

Embedded platforms, drivers and board design

Prerequisites Modules 17-18 and 31-35. Physical work remains optional and supervised.

Four lessons

36.1 Serial interfaces and peripheral contracts

Choose a communication interface using timing and electrical constraints.

36.2 Firmware, RTOS scheduling and driver lifecycle

Bound embedded work and make device state transitions explicit.

36.3 Schematics, PCB layout, power integrity and signal integrity

Translate a circuit into a manufacturable layout without ignoring physics.

36.4 Bring-up, instrumentation and fault isolation

Turn an unresponsive device into a sequence of testable hypotheses.

Module project

Design a simulated controller or peripheral with firmware, a host interface and a schematic-level review. A physical board extension begins only after the electrical, layout and bring-up plans receive qualified review.

Core references

R36 KiCad project — KiCad documentation

R51 Linux kernel contributors — Driver implementer API guide

R53 Zephyr Project — Zephyr documentation

R30 MIT OpenCourseWare — 6.002 Circuits and Electronics

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 36.1 • MODULE 36

Serial interfaces and peripheral contracts

Learning goal Choose a communication interface using timing and electrical constraints.

UART, SPI and I2C offer different ways to exchange information. UART communication depends on agreed framing and baud timing without a shared clock. SPI commonly uses a controller-provided clock and device-select signals, with mode and timing choices. I2C uses addressed devices and shared lines with electrical and protocol requirements such as pull-ups. These names do not specify every device-specific command or voltage level.

A robust driver documents pin assignment, voltage domain, clock limits, transfer ordering, error conditions and device register semantics. USB and PCIe add more complex enumeration and protocol layers; use supported controllers and documented software interfaces rather than treating them as generic serial wires. Validate a protocol model before attaching physical devices.

WORKED EXAMPLE

Two SPI endpoints can use the same bit rate but disagree on clock phase, sampling the wrong edge. Matching only the frequency is not enough to establish compatibility.

PRACTICE + PASS CRITERION

Build a simulated peripheral transaction decoder and driver for one interface. Pass: incorrect framing or mode is detected, timeouts are bounded and a complete interface-contract table is provided.

ASSIGNED STUDY

R53: device drivers, UART, SPI and I2C APIs; R51: bus-level driver documentation. For hardware, add the exact peripheral datasheet.

CHECK YOUR UNDERSTANDING

1. Why can two SPI devices fail at the same clock rate?

2. Does an interface name guarantee voltage compatibility?

3. Why begin with one interface rather than every bus?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 36.2 • MODULE 36

Firmware, RTOS scheduling and driver lifecycle

Learning goal Bound embedded work and make device state transitions explicit.

Firmware initializes hardware, configures peripherals and runs application behavior under platform constraints. A real-time operating system provides scheduling and synchronization mechanisms, but deadlines still depend on task design, interrupt load and resource contention. Assign responsibilities to interrupt context, worker threads and periodic tasks deliberately.

A driver lifecycle includes initialization, use, suspend/resume, removal and failure. Device state may disappear while a request is pending. Buffer ownership and cancellation must remain correct across those transitions. Watchdogs detect certain failures only when serviced meaningfully; repeatedly feeding a watchdog from a stuck loop defeats the intended supervision. Kernel experiments belong in disposable or simulated environments, not a primary computer.

WORKED EXAMPLE

A high-priority input task that waits on a low-priority logging task can miss its deadline despite using an RTOS. Moving logging off the critical path addresses the structural problem.

PRACTICE + PASS CRITERION

Use a supported RTOS simulator or a deterministic scheduler model to implement input, processing and logging tasks. Pass: deadlines are evaluated under injected load and removal/cancellation leaves no live invalid requests.

ASSIGNED STUDY

R53: kernel scheduling, synchronization and device model; R51: driver lifecycle and power management.

CHECK YOUR UNDERSTANDING

1. Does an RTOS automatically make all tasks meet deadlines?

2. Why not feed a watchdog unconditionally from a stuck loop?

3. What must happen to outstanding requests when a device disappears?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 36.3 • MODULE 36

Schematics, PCB layout, power integrity and signal integrity

Learning goal Translate a circuit into a manufacturable layout without ignoring physics.

A schematic specifies electrical connectivity and component intent; a PCB layout determines physical geometry and interconnect. Footprint pin numbering, connector orientation and power-domain assignment must match actual parts. Electrical-rule and design-rule checks catch some mistakes, but they cannot prove the circuit is sensible or safe.

Fast edges make traces and return paths important even when the logical clock seems modest. Signal integrity concerns waveform quality, impedance, reflections and crosstalk; power integrity concerns supply stability under changing current demand. Decoupling, plane continuity, stackup and placement are interconnected choices. High-speed memory and serial links need deeper analysis and qualified review. Start with a low-speed educational design, not a first-attempt graphics card.

WORKED EXAMPLE

A correct net name does not guarantee a usable signal if the physical return path is badly interrupted. Conversely, a visually tidy board can still have swapped connector pins or incompatible voltage domains.

PRACTICE + PASS CRITERION

Create a low-speed peripheral schematic and unrouted or simple two-layer layout in KiCad. Pass: footprints are checked against datasheets, ERC/DRC findings are resolved or justified and a reviewer inspects power and return paths before fabrication.

ASSIGNED STUDY

R36: schematic editor, PCB editor, ERC and DRC; R30: nonideal circuit behavior; R49: interconnect and packaging extension.

CHECK YOUR UNDERSTANDING

1. Does passing DRC prove electrical correctness?

2. Why do return paths matter?

3. Why verify footprints against datasheets?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 36.4 • MODULE 36

Bring-up, instrumentation and fault isolation

Learning goal Turn an unresponsive device into a sequence of testable hypotheses.

Bring-up checks power, clocks, reset, communication and function in an intentional order. A known-good firmware image and a minimal heartbeat separate basic execution from the full application. Logic analyzers observe digital timing under their bandwidth and threshold limits; oscilloscopes reveal analog waveform behavior but require safe probing and interpretation.

Use current-limited approved low-voltage supplies only under a reviewed plan for physical work. Do not probe mains or modify high-energy power systems. Record expected observations before powering hardware and stop when results exceed safe limits. A fault tree helps separate software, interface and electrical hypotheses. Preserve evidence and change one variable at a time rather than repeatedly reflashing and rewiring at random.

WORKED EXAMPLE

If no serial output appears, verify the expected voltage domain, correct pins, configured baud rate, reset state and program execution before assuming the application algorithm is wrong.

PRACTICE + PASS CRITERION

Write and rehearse a bring-up procedure against a simulated device with injected clock, reset and protocol faults. Pass: each fault has a safe diagnostic branch and the procedure includes explicit stop conditions.

ASSIGNED STUDY

R53: board support and debugging; R36: fabrication outputs and design review; selected instrument manuals for any physical extension.

CHECK YOUR UNDERSTANDING

1. Why test power, clock and reset before complex application behavior?

2. What is the value of an expected-observation table?

3. What should happen when a physical observation exceeds a reviewed safe limit?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 37

GPU microarchitecture and memory systems

Prerequisites Modules 16, 22, 32 and 34.

Four lessons

37.1 SIMT execution, divergence, occupancy and latency hiding

Explain why many threads do not guarantee useful parallel work.

37.2 Coalescing, local storage, caches and shared-memory banks

Match data layout to the memory transactions actually performed.

37.3 Raster, texture, matrix and ray-tracing units

Understand fixed-function acceleration as specialized tradeoffs.

37.4 CPU-GPU sharing, discrete memory and synchronization

Treat shared addressability separately from ownership and cost.

Module project

Create a GPU architecture workbook connecting shader behavior to execution groups, registers, memory transactions and fixed-function resources. Measure one accessible GPU or clearly label a simulator-only study.

Core references

R37 NVIDIA — CUDA Programming Guide

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

R43 Samuel Williams, Andrew Waterman and David Patterson — Roofline performance model, technical report

R21 Khronos Group — Vulkan tutorial

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 37.1 • MODULE 37

SIMT execution, divergence, occupancy and latency hiding

Learning goal Explain why many threads do not guarantee useful parallel work.

GPUs organize work into groups that execute under architecture-specific scheduling rules. SIMT exposes individual threads while grouping their instruction execution. Divergent control flow can require different paths to be executed for different active lanes, reducing useful work. Exact group size and behavior depend on the target architecture and API capabilities.

Occupancy describes how much execution state is resident relative to a hardware limit, not the fraction of arithmetic units doing useful work. Registers, shared memory and block size can constrain residency. More resident work can help hide latency, but maximizing occupancy may increase spills or reduce per-thread efficiency. Profile the actual kernel and distinguish occupancy, utilization and achieved throughput.

WORKED EXAMPLE

A group with half its lanes taking an expensive branch and half taking another may perform both paths with different active masks. The cost is not necessarily the average of the two branches.

PRACTICE + PASS CRITERION

Compare coherent and divergent workloads, varying register pressure or group size where supported. Pass: outputs agree, device group assumptions are recorded and the report does not equate occupancy with performance.

ASSIGNED STUDY

R37: programming model, execution and hardware implementation; R48: vector and multithreaded architecture foundations.

CHECK YOUR UNDERSTANDING

1. What is branch divergence in grouped GPU execution?

2. Does high occupancy guarantee high throughput?

3. Why can reducing register use sometimes hurt?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 37.2 • MODULE 37

Coalescing, local storage, caches and shared-memory banks

Learning goal Match data layout to the memory transactions actually performed.

Adjacent threads accessing adjacent locations can allow efficient grouped memory transactions. Strided or scattered access may require more transactions for the same useful bytes. Alignment, element size and architecture rules influence the result. Do not infer transaction count solely from source-code load count.

On-chip shared storage can enable reuse and communication within a group, but its banks and synchronization requirements matter. Bank conflicts can serialize accesses under the hardware's rules. Caches help some patterns and not others; a cache hit also has a finite cost. Data-layout changes must preserve semantics and include conversion overhead when comparing end-to-end performance.

WORKED EXAMPLE

If a group needs neighboring float values but the structure stores a large unrelated record between them, the useful bytes can be spread across many transactions. A structure-of-arrays layout may reduce wasted traffic for that access pattern.

PRACTICE + PASS CRITERION

Benchmark contiguous, strided and tiled access with identical useful work. Pass: count useful bytes, state the relevant memory level and include any layout-conversion cost in the application comparison.

ASSIGNED STUDY

R37: device memory access, shared memory and performance considerations; R43: operational intensity.

CHECK YOUR UNDERSTANDING

1. What does coalescing improve?

2. Why can shared memory still cause stalls?

3. Why include layout conversion in a comparison?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 37.3 • MODULE 37

Raster, texture, matrix and ray-tracing units

Learning goal Understand fixed-function acceleration as specialized tradeoffs.

GPUs contain resources specialized for operations such as rasterization, texture filtering or particular matrix arithmetic; some also accelerate parts of ray tracing. The presence, capability and limits of these units vary. A shader can become limited by a specialized unit rather than general arithmetic.

Specialization trades flexibility for efficiency. Supported formats, tile shapes, precision and data movement determine whether an operation maps well. Ray-tracing acceleration does not remove shader work, scene-build cost or memory traffic. Matrix throughput figures are meaningful only with their stated data types and operation-count conventions. Compare equivalent numerical quality and complete work, not incompatible headline peaks.

WORKED EXAMPLE

A fast low-precision matrix path may require packing, conversion and an accuracy check. For a tiny matrix, those costs can outweigh the accelerated arithmetic.

PRACTICE + PASS CRITERION

Analyze one texture-heavy, arithmetic-heavy or ray-query workload. Pass: identify the likely resource limits using measurements or a documented model and state supported precision and feature assumptions.

ASSIGNED STUDY

R37: architecture and specialized operations; R21: graphics/compute capabilities; R23: ray intersection and acceleration reference.

CHECK YOUR UNDERSTANDING

1. Does a ray-tracing unit eliminate the rest of rendering work?

2. Why compare the same numerical precision or quality?

3. When might specialized hardware lose on a small workload?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 37.4 • MODULE 37

CPU-GPU sharing, discrete memory and synchronization

Learning goal Treat shared addressability separately from ownership and cost.

A discrete GPU may have memory reached through explicit transfers, while integrated or unified-memory systems expose different sharing arrangements. Shared addressing does not imply uniform access cost or automatic synchronization. Data migration, cache behavior and coherency rules depend on the platform and API.

Ownership still matters: the CPU must not modify data while the GPU consumes it unless the contract permits that access. DMA and cache maintenance may require specific operations. Pinned or mapped memory has tradeoffs and resource limits. Measure transfer, compute and synchronization separately, then report the end-to-end path. A portable engine should choose strategies by capabilities and workload rather than assuming every GPU behaves like one desktop card.

WORKED EXAMPLE

An integrated GPU and CPU can share physical memory yet contend for the same bandwidth. Eliminating an explicit copy does not create unlimited throughput.

PRACTICE + PASS CRITERION

Compare an upload/compute/readback path with an available shared-memory path, or model both with stated assumptions. Pass: correct ownership, separate phase timings and no claim that unified means zero-cost.

ASSIGNED STUDY

R37: memory management and synchronization; R21, R24 or R25: the selected platform's resource-sharing contract.

CHECK YOUR UNDERSTANDING

1. Does shared addressing mean zero data-access cost?

2. Why is CPU modification during GPU use risky?

3. What must an end-to-end accelerator benchmark include?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 38

Quantitative performance engineering

Prerequisites Modules 09, 16, 18, 30 and 37.

Four lessons

38.1 Amdahl, roofline and bottleneck models

Use upper bounds to reject implausible optimization claims.

38.2 Profilers, counters and trustworthy benchmark design

Distinguish observation from an explanation that still needs testing.

38.3 SIMD, cache blocking, NUMA and job scheduling

Optimize locality and parallelism together.

38.4 Energy, thermal behavior and sustained performance

Evaluate useful work over time rather than only a short peak.

Module project

Produce a performance investigation with a hypothesis, controlled workload, raw measurements, uncertainty, a bottleneck model and a verified optimization. A negative result is acceptable when the evidence is rigorous.

Core references

R43 Samuel Williams, Andrew Waterman and David Patterson — Roofline performance model, technical report

R37 NVIDIA — CUDA Programming Guide

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

R26 Epic Games — Programming with C++ in Unreal Engine

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 38.1 • MODULE 38

Amdahl, roofline and bottleneck models

Learning goal Use upper bounds to reject implausible optimization claims.

Amdahl's law limits total speedup when only part of a workload improves. For serial fraction s and ideal acceleration p of the remainder, speedup is 1/(s+(1-s)/p). This simplified model assumes the work decomposition remains valid and ignores new overhead unless added explicitly.

A roofline model bounds throughput by compute capacity and memory bandwidth times operational intensity. Define intensity at a particular memory level using actual data movement, not only source-level array sizes. The model is a ceiling, not a performance prediction or proof of the bottleneck. Latency, dependencies, occupancy and instruction mix can keep measured results below both roofs.

WORKED EXAMPLE

For a 1,000 GFLOP/s compute ceiling, 100 GB/s bandwidth and intensity 2 FLOP/byte, the bandwidth roof is 200 GFLOP/s. The simplified upper bound is min(1000,200)=200 GFLOP/s.

PRACTICE + PASS CRITERION

Build a roofline worksheet for one kernel and an Amdahl estimate for one frame optimization. Pass: units cancel correctly, modeled ceilings use the declared memory level and observed deviations are explained as hypotheses.

ASSIGNED STUDY

R43: original Roofline model; R48: quantitative architecture; revisit Module 18 parallel limits.

CHECK YOUR UNDERSTANDING

1. What is the example roofline bound?
2. What is the maximum speedup with 20% unchanged serial work and infinitely fast remaining work?
3. Does being below the roof prove the model is wrong?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 38.2 • MODULE 38

Profilers, counters and trustworthy benchmark design

Learning goal Distinguish observation from an explanation that still needs testing.

Hardware counters and traces expose events such as cache misses, stalls and executed work under tool-specific definitions. Counters can be multiplexed, sampled or unavailable. A metric name does not guarantee identical meaning across vendors or generations. Read the collection method and avoid treating one counter as a complete causal explanation.

Benchmark design controls workload, warmup, power state, compiler options and timing boundaries. Prevent dead-code elimination by validating outputs and making results observable. Repeated trials should be sufficiently independent for the statistical claim. Preserve raw data and record failed or anomalous runs with a declared exclusion policy. Compare medians or distributions appropriately rather than cherry-picking the best run.

WORKED EXAMPLE

A loop whose result is never used may be optimized away, making a timing appear spectacular. Checking a result checksum outside the measured region helps establish that the intended work occurred.

PRACTICE + PASS CRITERION

Create a benchmark harness with warmup, repeated runs, output validation and metadata. Pass: an intentional no-op or wrong-result variant is rejected and all included/excluded samples are documented.

ASSIGNED STUDY

R26: profiling; R37: performance analysis; R09: inference and uncertainty.

CHECK YOUR UNDERSTANDING

1. Why validate benchmark output?

2. Are similarly named counters identical across all processors?

3. Why declare an exclusion policy before inspecting results?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 38.3 • MODULE 38

SIMD, cache blocking, NUMA and job scheduling

Learning goal Optimize locality and parallelism together.

SIMD exploits regular operations across lanes, while threading distributes work across cores. Cache blocking reorganizes computation to reuse data before eviction. Each technique can alter numerical ordering and needs correctness tolerances or exactness requirements. Avoid undefined aliasing or out-of-bounds tricks to obtain a benchmark win.

Non-uniform memory access systems can make data placement and worker affinity important, but binding threads without measurement can also reduce flexibility. Job granularity balances scheduling overhead against parallel opportunities and load balance. False sharing can cause cache-coherence traffic even when threads modify different logical variables. Optimize the actual data ownership pattern, not only the number of threads.

WORKED EXAMPLE

A job requiring 1 microsecond of useful work with 2 microseconds of scheduling overhead is a poor unit of work under that model. Batching can improve efficiency, but overly large batches can create an imbalanced tail.

PRACTICE + PASS CRITERION

Optimize a particle or transform workload using one locality change and one scheduling change. Pass: test tail sizes, report numerical differences, and measure small as well as large workloads to expose overhead.

ASSIGNED STUDY

R37: parallel memory patterns; R48: multiprocessors and memory systems; R11: safe C++ and concurrency.

CHECK YOUR UNDERSTANDING

1. What is false sharing?

2. Why can very small jobs be inefficient?

3. Why test small workloads after a parallel optimization?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 38.4 • MODULE 38

Energy, thermal behavior and sustained performance

Learning goal Evaluate useful work over time rather than only a short peak.

Power is an instantaneous rate of energy use; energy is its integral over time. A design consuming more power can still use less energy for a task if it finishes sufficiently sooner, while sustained operation can be limited by thermal conditions. Performance-per-watt claims need a measurement boundary and a workload definition.

Short benchmark bursts may not represent steady operation after temperature and frequency settle. Record ambient or available temperature conditions, duration and power source without overriding safety protections. Software counters estimate power differently from external instruments; label the method and uncertainty. Dynamic quality scaling can preserve experience within a thermal budget, but should avoid distracting oscillation or violating simulation correctness.

WORKED EXAMPLE

Ignoring idle transitions, a task at 50 W for 2 seconds uses 100 J, while 80 W for 1 second uses 80 J. Lower instantaneous power is not automatically lower task energy.

PRACTICE + PASS CRITERION

Run a sustained workload within normal device limits or analyze a synthetic power trace. Pass: distinguish watts from joules, report the measurement boundary and compare early versus sustained performance.

ASSIGNED STUDY

R48: quantitative architecture; R51: device power management; R37: GPU execution and performance context.

CHECK YOUR UNDERSTANDING

1. How much energy is 50 W for 2 seconds?

2. Which example uses less task energy?

3. Why not disable thermal protections for a curriculum benchmark?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 39

Gaming-platform constraints and interactive latency

Prerequisites Modules 22, 27, 29 and 36-38.

Four lessons

39.1 Input-to-display latency and frame pacing

Account for the whole interaction path rather than only frame rate.

39.2 Mobile and handheld systems: memory, power and lifecycle

Design for suspension, limited budgets and sustained use.

39.3 XR spaces, stereo views, pose timing and comfort constraints

Render from runtime-provided timing and coordinate contracts.

39.4 Portability, platform services and console-development boundaries

Build transferable systems without assuming proprietary access.

Module project

Create a platform capability and latency report for the engine. Implement one non-desktop constraint in simulation or on an available device: touch input, suspend/resume, restricted memory, variable presentation or XR timing.

Core references

R22 SDL project — SDL3 documentation

R24 Microsoft — Direct3D 12 graphics

R25 Apple — Metal developer resources

R29 Microsoft — Xbox Accessibility Guidelines

R52 Khronos Group — OpenXR registry and specifications

R53 Zephyr Project — Zephyr documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 39.1 • MODULE 39

Input-to-display latency and frame pacing

Learning goal Account for the whole interaction path rather than only frame rate.

Input-to-display latency includes sensing, device polling, event delivery, simulation scheduling, rendering, queues, scanout and display response. These stages can overlap and their timing depends on when an input arrives. A high average frame rate is not a complete latency measurement. Distinguish throughput, frame pacing and the age of the displayed input state.

Buffering can improve utilization while increasing latency. Synchronization and presentation modes affect tearing, pacing and queue depth under platform-specific rules. Variable refresh can help presentation timing within supported ranges but does not remove CPU or GPU work. Measure an end-to-end path with appropriate instrumentation or clearly label a timing model as an estimate.

WORKED EXAMPLE

At 60 Hz, one full frame interval is about 16.67 ms. Two extra queued frame intervals can add roughly 33.3 ms of delay in a simplified model, even though the output still averages 60 frames per second.

PRACTICE + PASS CRITERION

Instrument input receipt, simulation consumption, render submission and presentation-related timestamps. Pass: explain which stages are actually measured and which remain estimated, and show a frame-pacing distribution.

ASSIGNED STUDY

R22: input and event timing; R24 or R25: presentation; R52: frame timing as an advanced comparison.

CHECK YOUR UNDERSTANDING

1. Is 60 frames per second a complete latency measurement?

2. What is two 60 Hz frame intervals?

3. Why label timestamp boundaries precisely?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 39.2 • MODULE 39

Mobile and handheld systems: memory, power and lifecycle

Learning goal Design for suspension, limited budgets and sustained use.

Mobile and handheld devices impose different input, memory, thermal and lifecycle constraints from an always-active desktop process. An application may lose focus, suspend or need to restore resources. Save enough state under the platform contract without assuming unlimited time during shutdown. Touch interaction and small displays require different interface testing.

Tile-oriented rendering architectures and shared memory can change the cost of passes and attachments, but device behavior must be verified rather than inferred from a label. Sustained frame targets should respect power and temperature limits. Quality tiers can adjust rendering work while preserving game rules and readability. Test low-memory and interrupted-session behavior deliberately.

WORKED EXAMPLE

A renderer that reloads every texture synchronously after resume may cause a long stall or memory spike. Staged restoration with placeholders can provide a more controlled return to play.

PRACTICE + PASS CRITERION

Add suspend/resume simulation and a constrained memory mode. Pass: progress survives the documented lifecycle, resources are restored safely and quality changes do not alter authoritative simulation rules.

ASSIGNED STUDY

R25: platform graphics resources; R22: application lifecycle events; R29: text and input usability; R51: power-management concepts.

CHECK YOUR UNDERSTANDING

1. Why not assume shutdown always provides unlimited save time?

2. Why measure sustained rather than only initial performance?

3. What should rendering quality tiers preserve?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 39.3 • MODULE 39

XR spaces, stereo views, pose timing and comfort constraints

Learning goal Render from runtime-provided timing and coordinate contracts.

Extended-reality rendering combines multiple views with tracked poses and runtime-managed presentation. A pose belongs to a coordinate space and a time. Using a correct position from the wrong space or an outdated time can still produce visibly incorrect motion. View configuration, field of view and projection should come from the runtime contract rather than hard-coded assumptions.

Prediction and late-stage updates reduce some effects of tracking and rendering delay, but they do not eliminate errors or every source of discomfort. Foveated rendering allocates detail according to a policy and available capabilities, with quality and tracking implications. Keep a desktop simulated-view route for learning before purchasing a headset. Physical testing must be voluntary, brief and stopped when uncomfortable; no medical benefit is implied.

WORKED EXAMPLE

A head pose sampled for an earlier frame should not be described as the pose at display time. The runtime's predicted display timing establishes which estimate the application should use.

PRACTICE + PASS CRITERION

Implement two simulated camera views and a pose-time log, or a minimal OpenXR scene on available supported hardware. Pass: spaces and units are explicit, frame timing follows the runtime and loss of tracking has defined behavior.

ASSIGNED STUDY

R52: spaces, view configuration, frame loop and timing; R29: interaction and accessibility review.

CHECK YOUR UNDERSTANDING

1. Why is a pose incomplete without a space and time?

2. Should stereo projection be hard-coded for every headset?

3. Does prediction remove all latency or discomfort risk?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 39.4 • MODULE 39

Portability, platform services and console-development boundaries

Learning goal Build transferable systems without assuming proprietary access.

Portability starts with explicit contracts for input, files, threading, graphics, audio and lifecycle. A platform layer should isolate differences while allowing necessary capabilities to remain visible. A lowest-common-denominator abstraction can hide performance opportunities, while scattered platform conditionals can make maintenance fragile. Test the boundary with a second implementation or a mock.

Console development may involve access-controlled SDKs, agreements and platform-specific requirements. Public self-study cannot provide those permissions or substitute for official documentation. Learn transferable concepts through desktop, mobile or open hardware, then use authorized platform materials when available. Record dependency licenses and distribution constraints for each target. Do not use leaked SDKs or assume technical compatibility grants distribution rights.

WORKED EXAMPLE

A filesystem interface must distinguish persistent saves from packaged read-only assets. Mapping both to one desktop directory can conceal a portability problem until deployment.

PRACTICE + PASS CRITERION

Write a platform interface and capability matrix, then implement a second backend or strict mock. Pass: assumptions about paths, input and lifecycle are exposed, and proprietary requirements are marked unavailable until authorized.

ASSIGNED STUDY

R22, R24 and R25: platform interfaces; R29: cross-device usability; official authorized platform documentation only for restricted targets.

CHECK YOUR UNDERSTANDING

1. Why can a strict mock help portability?

2. Does public self-study grant access to proprietary console SDKs?

3. Why separate packaged assets from persistent saves?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 40

Hardware/software co-design capstone

Prerequisites Gates F and G; Modules 36-39.

Four lessons

40.1 Workload characterization and measurable requirements

Derive hardware goals from a real software operation.

40.2 Architecture alternatives, arithmetic precision and dataflow

Explore several designs before committing to one.

40.3 Interface design, virtualization and end-to-end evaluation

Include the host software in the hardware result.

40.4 Design review, evidence tiers and undergraduate CE completion

Defend a co-design decision with limitations and reproducible artifacts.

Module project

Gate H: choose one game workload and compare a software baseline with a specialized hardware or architecture alternative. Deliver a reference implementation, model or RTL, software interface, verification evidence and an end-to-end tradeoff report.

Core references

R43 Samuel Williams, Andrew Waterman and David Patterson — Roofline performance model, technical report

R48 MIT OpenCourseWare — 6.823 Computer System Architecture

R49 Christopher Batten / Cornell University — ECE5745 Complex Digital ASIC Design

R38 gem5 project — gem5 documentation

R33 YosysHQ — Yosys documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 40.1 • MODULE 40

Workload characterization and measurable requirements

Learning goal Derive hardware goals from a real software operation.

Co-design begins with a workload, not a favorite chip block. Identify input sizes, data distributions, frequency of use, latency needs, quality constraints and failure behavior. Measure the baseline and determine whether the operation limits the complete application. A subsystem with an attractive microbenchmark may be irrelevant to the frame critical path.

Requirements should include throughput, worst supported latency, numerical error, storage, interface costs and maintainability. Separate hard constraints from preferences. Choose representative and adverse inputs, keeping a held-out set for final evaluation. A successful design may be one that rejects unnecessary hardware after showing that a software improvement is cheaper and sufficient.

WORKED EXAMPLE

Accelerating a 0.2 ms operation in a 20 ms frame can save at most 1% of frame time under a simple additive model. A larger memory or synchronization bottleneck may deserve attention first.

PRACTICE + PASS CRITERION

Select a workload from the engine and write a requirement sheet plus baseline measurements. Pass: each target has units and a test, and the maximum plausible application benefit is estimated.

ASSIGNED STUDY

R43: bottleneck modeling; R48: quantitative design; R49: project methodology.

CHECK YOUR UNDERSTANDING

1. Why start with workload measurements?

2. What maximum fraction of the example frame can removing 0.2 ms save?

3. Can rejecting a hardware proposal be a successful outcome?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 40.2 • MODULE 40

Architecture alternatives, arithmetic precision and dataflow

Learning goal Explore several designs before committing to one.

A design space includes parallel lanes, pipeline depth, local storage, data reuse, arithmetic precision and the boundary between hardware and software. Each choice affects more than one metric. Wider parallelism can demand more memory bandwidth; deeper pipelines can improve frequency while increasing latency and control complexity.

Quantization requires an error budget tied to the application. Test saturation, rounding and extreme values against a reference. A dataflow diagram should show where values are produced, reused and transferred. Analytical models help eliminate infeasible candidates before expensive implementation. Record assumptions and compare a simple baseline with at least two alternatives rather than presenting one design as inevitable.

WORKED EXAMPLE

Doubling arithmetic lanes does not double useful throughput if the memory interface can supply only the original amount of data and the computation has no additional reuse.

PRACTICE + PASS CRITERION

Model three variants, such as scalar, pipelined and multi-lane. Pass: resource and bandwidth estimates are explicit, fixed-point error meets a chosen application tolerance and rejected variants have documented reasons.

ASSIGNED STUDY

R49: architecture and RTL modeling; R43: bandwidth limits; R33: resource inference.

CHECK YOUR UNDERSTANDING

1. Why might twice as many lanes fail to double throughput?

2. What should determine acceptable quantization error?

3. Why explore more than one architecture?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 40.3 • MODULE 40

Interface design, virtualization and end-to-end evaluation

Learning goal Include the host software in the hardware result.

The host interface specifies work descriptors, buffers, alignment, ownership, completion and errors. Long-running or shared accelerators also need fairness, isolation and recovery policy. A malformed descriptor must not grant arbitrary access to unrelated memory. Use a simulated device or established protected driver framework for educational experiments.

End-to-end evaluation includes packaging data, transfer, queueing, computation, synchronization and result consumption. Pipeline overlap can improve throughput without equally reducing one request's latency. Measure at realistic batch sizes and include a fallback path. Hardware that works only when every input is perfect is not an integrated engineering result.

WORKED EXAMPLE

A kernel with a 5x arithmetic speedup may deliver only 1.2x application speedup after transfers and setup. Both numbers can be correct if their boundaries are stated.

PRACTICE + PASS CRITERION

Implement a host driver model with invalid-descriptor, timeout and reset tests. Pass: buffer bounds are enforced, ownership survives failure and results include both kernel and end-to-end timing.

ASSIGNED STUDY

R51: DMA and device interfaces; R49: system integration; R38: modeled systems when hardware is unavailable.

CHECK YOUR UNDERSTANDING

1. Why validate device work descriptors?

2. Can kernel and application speedups legitimately differ?

3. Why test realistic batch sizes?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 40.4 • MODULE 40

Design review, evidence tiers and undergraduate CE completion

Learning goal Defend a co-design decision with limitations and reproducible artifacts.

A design review connects requirements to evidence: functional results, numerical quality, timing, resources, integration and risk. Label measurements according to their source. An analytical estimate, cycle-accurate model, synthesized netlist and physical instrument reading support different claims. Device-specific timing or area should not be generalized without a justified model.

The final comparison needs identical workloads, a competent baseline and disclosure of failed cases. Discuss cost beyond arithmetic, including tool dependence, verification effort, portability and maintenance. Preserve source, scripts, raw data and exact configuration. An independent reviewer should be able to repeat the central experiment and challenge at least one assumption. This checkpoint prepares the learner for research-level specialization, not independent production-chip signoff.

WORKED EXAMPLE

A synthesis estimate from an educational library is useful for comparing variants under that setup. It is not a vendor quotation or a guarantee of commercial silicon performance.

PRACTICE + PASS CRITERION

Complete Gate H and a design-review presentation. Pass: every major claim traces to an artifact, the baseline is reproduced and a limitations section clearly separates educational estimates from production claims.

ASSIGNED STUDY

R49: design-project reporting; R48: architectural tradeoffs; revisit the evidence and project rubrics in the handbook.

CHECK YOUR UNDERSTANDING

1. Why label simulation and physical measurements separately?

2. What makes a baseline competent?

3. Does Gate H authorize production chip fabrication?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.